

CSC1003 Tutorial 8

Programming Efficiency

Liang Yuyang

119010174@link.cuhk.edu.cn

November 26, 2025

Outline

- 1 Why Efficiency Matters
- 2 Evaluating Efficiency
- 3 The Core Tool: Big-O Notation
- 4 Case Studies
- 5 Conclusion

Why Care About Efficiency?

The Problem of Scale

- **Intuition:** Computers are fast, does efficiency still matter?
- **Reality:** Yes, more than ever, due to **data volume**.

The Challenge of Scale

- Modern data sets are massive (e.g., Google served 30,000,000,000,000 pages, covering 100,000,000 GB - how long to search brute force?).
- Small inefficiencies lead to massive time costs when input size (N) is huge.
- **Goal:** Assess an algorithm's **Scalability** → how its performance changes as N grows.

Two Dimensions of Efficiency

Time vs. Space Trade-off

- **Time Efficiency:** The time required for a program to run.
- **Space Efficiency:** The memory or storage space required.

The Core Trade-off

- Time and space often constrain each other.
- We often **trade space for time** (e.g., using memory for speed).
- **Example:** Memoization (e.g., Fibonacci) → Stores results to avoid recalculation, increasing space for better time efficiency.

Method I: Timing a Program

Using a System Timer

- **Approach:** Use a system timer (time module) to measure the wall-clock execution time.
- **Code Snippet:**

```
1 import time
2 t0 = time.time()
3 function_call(input)
4 t1 = time.time()
5 print("Time:", t1 - t0, "s")
```

Listing 1: Timing Code

Why Timing is Insufficient

- Results are **unreliable** and **non-general**.
- Depends on hardware, OS, and specific implementation.
- Fails to establish a mathematical relationship between N and time.

Method II: Counting Operations

A Step Towards Abstraction: Code Example

- **Approach:** We count the number of basic operations as a function of the input size N .
 - Mathematical Operation
 - assignment
 - comparison
 - accessing objects in memory
- **Assumption:** Each basic operation takes a constant amount of time.

Example: mysum(x)

```
def mysum(x):  
    total = 0  
    # loop x times  
    for i in range(x):  
        total = total + i  
    return total
```

Listing 2: Summing integers from 0 to x-1

Method II: Counting Operations (Analysis)

Deriving the Time Function $T(N)$

Detailed Operation Count ($T(x)$):

- ① `total = 0`: 1 op (Assignment)
- ② `for i in range(x)`: Loop runs x times
- ③ `total = total + i` (inside loop):
 - Accessing `i` (1 op)
 - Addition (1 op)
 - Assignment to `total` (1 op)
 - Total: $3 \text{ ops} \times x \text{ times}$
- ④ `return total`: 1 op (Return)

Total Time Function:

- $T(x) = 1 \text{ (init)} + (3 \times x) \text{ (loop)} + 1 \text{ (return)}$
- $T(x) = 3x + 2$

The Conclusion

- **Advantage:** Establishes a clear functional relationship $T(N)$.
- **Disadvantage:** Too dependent on implementation details (Is addition always 1 op?).

This leads us to a purely **abstract** method: Big-O Notation.

Input Cases: Best, Worst, Average

Defining the Scope of Performance

search_for_elmt(L, e):

```
def search_for_elmt(L, e):  
    for i in range(len(L)):  
        if L[i] == e:  
            return i # Best Case: found at index i  
    return -1 # Worst Case: element not found
```

Listing 3: Linear Search Pseudocode

Case	Requirement
Best Case	Element e is the first.
Worst Case	Element e is absent/last.
Average Case	Lookup of about half the elements.

- **Focus:** We concentrate on the **Worst Case** as it represents the algorithm's performance limit and reliability.

Introducing Big-O Notation $\mathcal{O}()$

The Measure of Asymptotic Growth

- **Definition:** $\mathcal{O}(f(n))$ measures the **asymptotic upper bound** of an algorithm's running time $T(n)$.
- It describes the rate of growth as the input size N approaches infinity ($N \rightarrow \infty$).
- Big-O describes the **Algorithm**, independent of machine or specific implementation.

Two Key Simplification Principles

- 1 **Focus on the Dominant Term:** Only keep the term that grows fastest.
- 2 **Ignore Multiplicative Constants:** Constants are irrelevant for large N .

Big-O Notation Examples

Applying the Simplification Principles

Original Time $T(n)$	Dominant Term	Big-O $\mathcal{O}()$
$n^2 + 2n + 2$	n^2	$\mathcal{O}(n^2)$
$n^2 + 10^5 n + 3^{1000}$	n^2	$\mathcal{O}(n^2)$
$\log n + n + 4$	n	$\mathcal{O}(n)$
$0.0001n \log n + 300n$	$n \log n$	$\mathcal{O}(n \log n)$
$2n^{30} + 3^n$	3^n	$\mathcal{O}(3^n)$

- **Growth Hierarchy:**

$$\mathcal{O}(1) < \mathcal{O}(\log n) < \mathcal{O}(n) < \mathcal{O}(n \log n) < \mathcal{O}(n^2) < \mathcal{O}(2^n).$$

Rules for Combining Big-O I: Addition

Sequential Statements

The Addition Rule

$O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$. The total complexity is determined by the slowest step.

Example:

```
# Part 1:  $O(n)$ 
for i in range(n):
    operation_A()

# Part 2:  $O(n^2)$ 
for j in range(n * n):
    operation_B()
```

- **Calculation:** $O(n) + O(n^2) = O(n + n^2) = O(n^2)$.

Rules for Combining Big-O II: Multiplication

Nested Statements

The Multiplication Rule

$O(f(n)) \times O(g(n)) = O(f(n) \cdot g(n))$. The total complexity is the product of the complexities of the nested steps.

Example: Nested Loops

```
# Outer loop:  $O(n)$ 
for i in range(n):
    # Inner loop:  $O(n)$ 
    for j in range(n):
        print('Nested Operation')
```

- **Calculation:** $O(n) \times O(n) = O(n^2)$.

Case Study: Linear $\mathcal{O}(N)$

Complexity Proportional to Input Size

- **Example:** Summing the digits of a string s .

```
1 def addDigits(s):  
2     val = 0  
3     for c in s:  
4         val += int(c)  
5     return val
```

Analysis

- The loop executes once for every character in the string s .
- If $N = \text{len}(s)$, the time is proportional to N .
- **Complexity:** $\mathcal{O}(N)$.

Case Study: Quadratic $\mathcal{O}(N^2)$

Complexity Proportional to the Square of Input Size

- **Example:** Finding the intersection of two lists, L1 and L2.

```
1 def intersect(L1, L2):  
2     tmp = []  
3     for e1 in L1:                # Outer loop: N steps  
4         for e2 in L2:            # Inner loop: N steps  
5             if e1 == e2:  
6                 tmp.append(e1)  
7     return tmp
```

Analysis

- Assuming $\text{len}(L1) \approx \text{len}(L2) \approx N$.
- The comparison is performed $N \times N = N^2$ times.
- **Complexity:** $\mathcal{O}(N^2)$.

This could be optimized to $\mathcal{O}(N)$ using a Hash Set/Dictionary for $\mathcal{O}(1)$ lookups. The choice of **data structure** matters greatly!

Summary and Next Steps

- Evaluate algorithm efficiency abstractly using **Big-O Notation** for **Scalability**.
- Focus on the **Worst Case** and the **Dominant Term**.
- Goal: Design algorithms with the lowest possible Order of Growth to handle massive inputs.
- **Next:** Applying $\mathcal{O}()$ to analyze common searching and sorting algorithms.

Thank You!